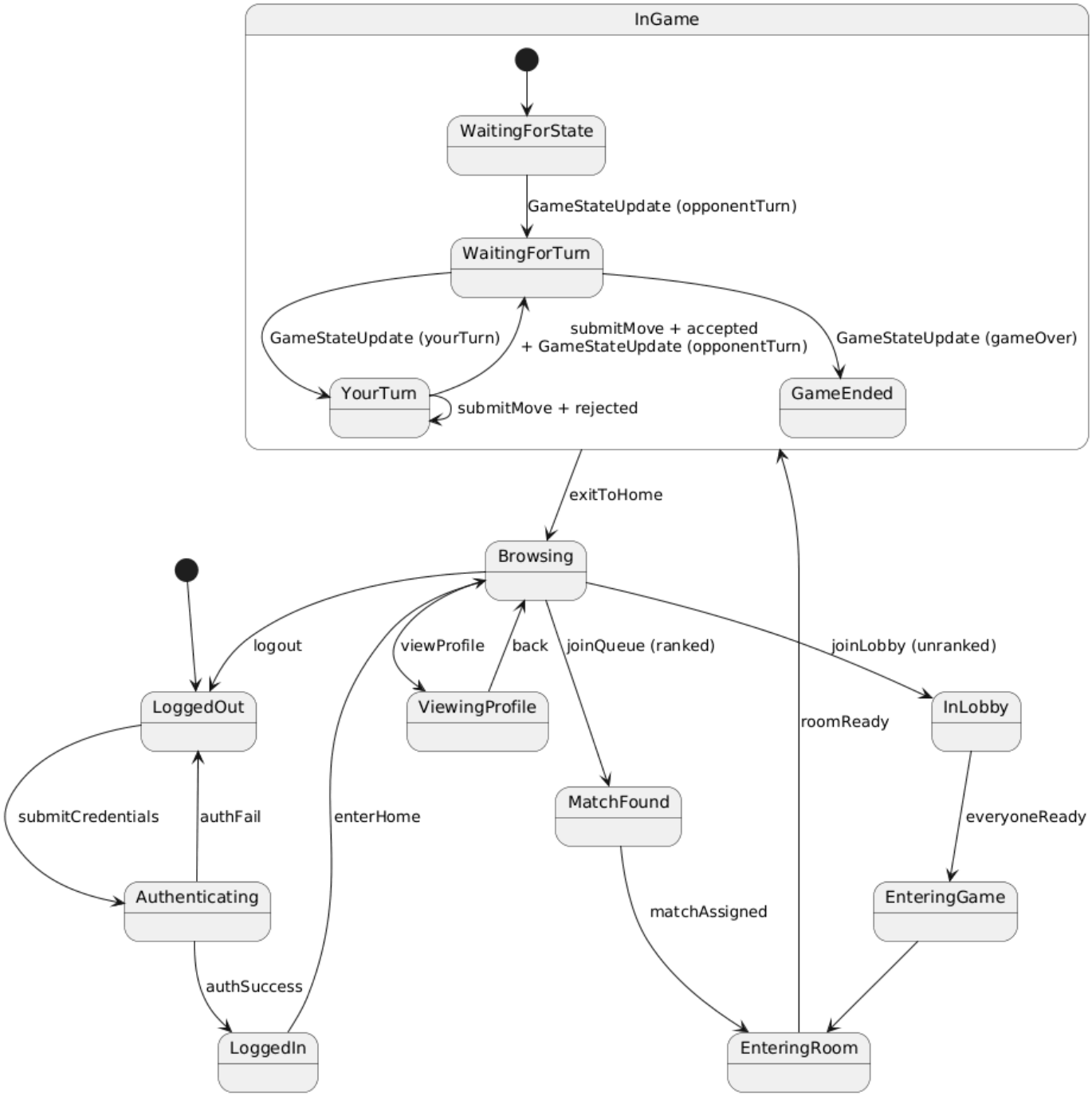
# State Machine Overview

## SENG300 | Online Multiplayer Game Platform — Project Iteration 1

This document provides a structured overview of all state machines defined for the platform. Six diagrams are organized across three layers: the platform session layer (what the player experiences navigating the system), the game session layer (how a single game room is managed), and the game logic layer (how individual game types track their internal state).

## Diagram 01 — Player Session State Machine

**File:** `01.03 Player SM.png`




## Purpose

Tracks the full lifecycle of a single player's interaction with the platform from first launch through authentication, navigation, matchmaking, and gameplay. This is the primary client-side session model.

## States

| State | Description |
|---|---|
| `LoggedOut` | Initial state. Player has no active session. |
| `Authenticating` | Player has submitted credentials; waiting for the auth stub to respond. |

| State | Description |
|---|---|
| `LoggedIn` | Credentials validated; session established. Transient — immediately proceeds to home. |
| `Browsing` | Central hub. Player is on the dashboard, able to navigate to all features. |
| `ViewingProfile` | Player is viewing their own profile and stats. |
| `MatchFound` | Matchmaking has paired the player with an opponent; awaiting assignment to a room. |
| `EnteringRoom` | Player has been assigned a room and is joining the game session. |
| `InLobby` | Player is in an unranked lobby, waiting for all players to ready up. |
| `EnteringGame` | All lobby players are ready; transitioning into the active game. |
| `InGame` *(composite)* | Composite state encapsulating the full in-game experience (see internal states below). |

## InGame Composite — Internal States

| State | Description |
|---|---|
| `WaitingForState` | Initial state on entering InGame. Waiting for the first `GameStateUpdate` to arrive. |
| `WaitingForTurn` | It is the opponent's turn. Player is idle, waiting for a `GameStateUpdate(yourTurn)`. |
| `YourTurn` | It is the player's turn. Player can submit a move. |
| `GameEnded` | Game has concluded. Waiting for the player to exit back to Browsing. |

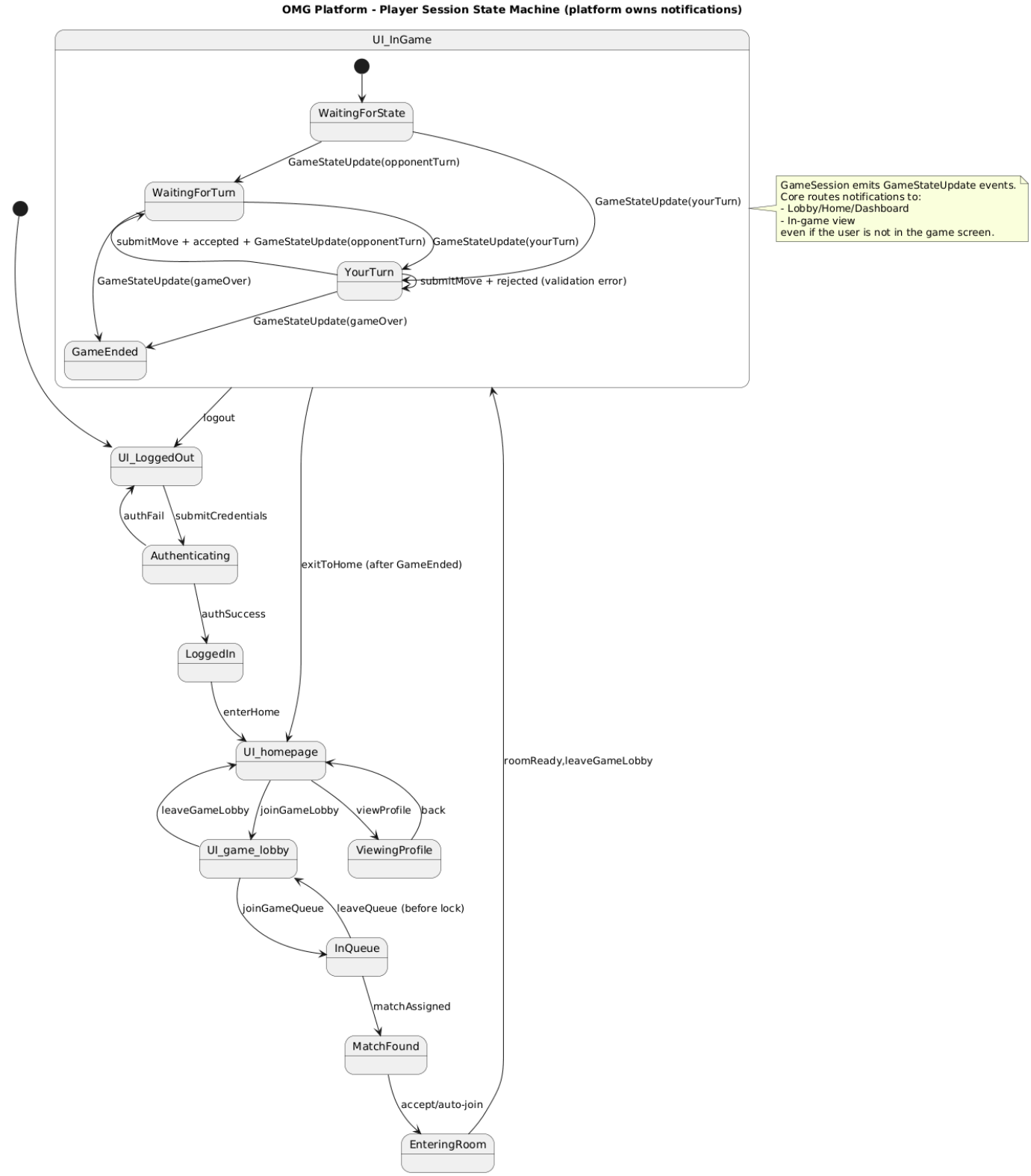
## Key Transitions

| From | Event | To |
|---|---|---|
| `LoggedOut` | `submitCredentials` | `Authenticating` |
| `Authenticating` | `authSuccess` | `LoggedIn` |
| `Authenticating` | `authFail` | `LoggedOut` |
| `LoggedIn` | `enterHome` | `Browsing` |
| `Browsing` | `logout` | `LoggedOut` |
| `Browsing` | `viewProfile` | `ViewingProfile` |
| `Browsing` | `joinQueue (ranked)` | `MatchFound` |
| `Browsing` | `joinLobby (unranked)` | `InLobby` |
| `ViewingProfile` | `back` | `Browsing` |
| `MatchFound` | `matchAssigned` | `EnteringRoom` |

| From | Event | To |
| --- | --- | --- |
| InLobby | everyoneReady | EnteringGame |
| EnteringRoom / EnteringGame | *(room ready)* | InGame |
| WaitingForState | GameStateUpdate(opponentTurn) | WaitingForTurn |
| WaitingForTurn | GameStateUpdate(yourTurn) | YourTurn |
| YourTurn | submitMove + rejected | YourTurn (self-loop) |
| YourTurn | submitMove + accepted + GameStateUpdate(opponentTurn) | WaitingForTurn |
| WaitingForTurn / YourTurn | GameStateUpdate(gameOver) | GameEnded |
| GameEnded | exitToHome | Browsing |

## Diagram 02 — Platform Player Session State Machine

**File:** `01.01_Platform_SM.png`




## Purpose

A more detailed version of the Player Session SM from the platform's architectural perspective. This diagram emphasizes that `GameSession` emits `GameStateUpdate` events and that `PlatformCore` routes notifications to wherever the player currently is — lobby, home, dashboard, or in-game view — regardless of which screen the player is on.

### States (Platform View)

| State | Description |
|---|---|
| `UI_LoggedOut` | Player has no authenticated session. |
| `Authenticating` | Credentials submitted; awaiting auth response. |
| `LoggedIn` | Session established; transient before home navigation. |
| `UI_homepage` | Dashboard home. Player navigates to lobby, profile, or queue from here. |
| `ViewingProfile` | Player viewing their own profile. |
| `UI_game_lobby` | Player is inside a game lobby, able to join the queue or leave. |
| `InQueue` | Player is in the ranked matchmaking queue. |
| `MatchFound` | A match has been assigned; player is about to enter a room. |
| `EnteringRoom` | Player is joining the game session. |
| `UI_InGame` *(composite)* | Full in-game composite state (same internal structure as Diagram 01). |

### Key Differences from Diagram 01

- States are prefixed `UI_` to emphasize these are UI-layer states, not internal server states.
- The path from `UI_homepage` to the game is more explicit: `UI_game_lobby` → `InQueue` → `MatchFound` → `EnteringRoom`, versus Diagram 01's shorter `Browsing` → `MatchFound` path.
- `leaveQueue (before lock)` is an explicit transition from `InQueue` back to `UI_game_lobby`, clarifying that queue exit is only possible before a match is locked.
- `roomReady, leaveGameLobby` is a direct transition from `UI_homepage` to `UI_InGame`, supporting the unranked lobby path.
- `exitToHome (after GameEnded)` is explicitly labeled to clarify this transition only fires after the game ends, not mid-session.

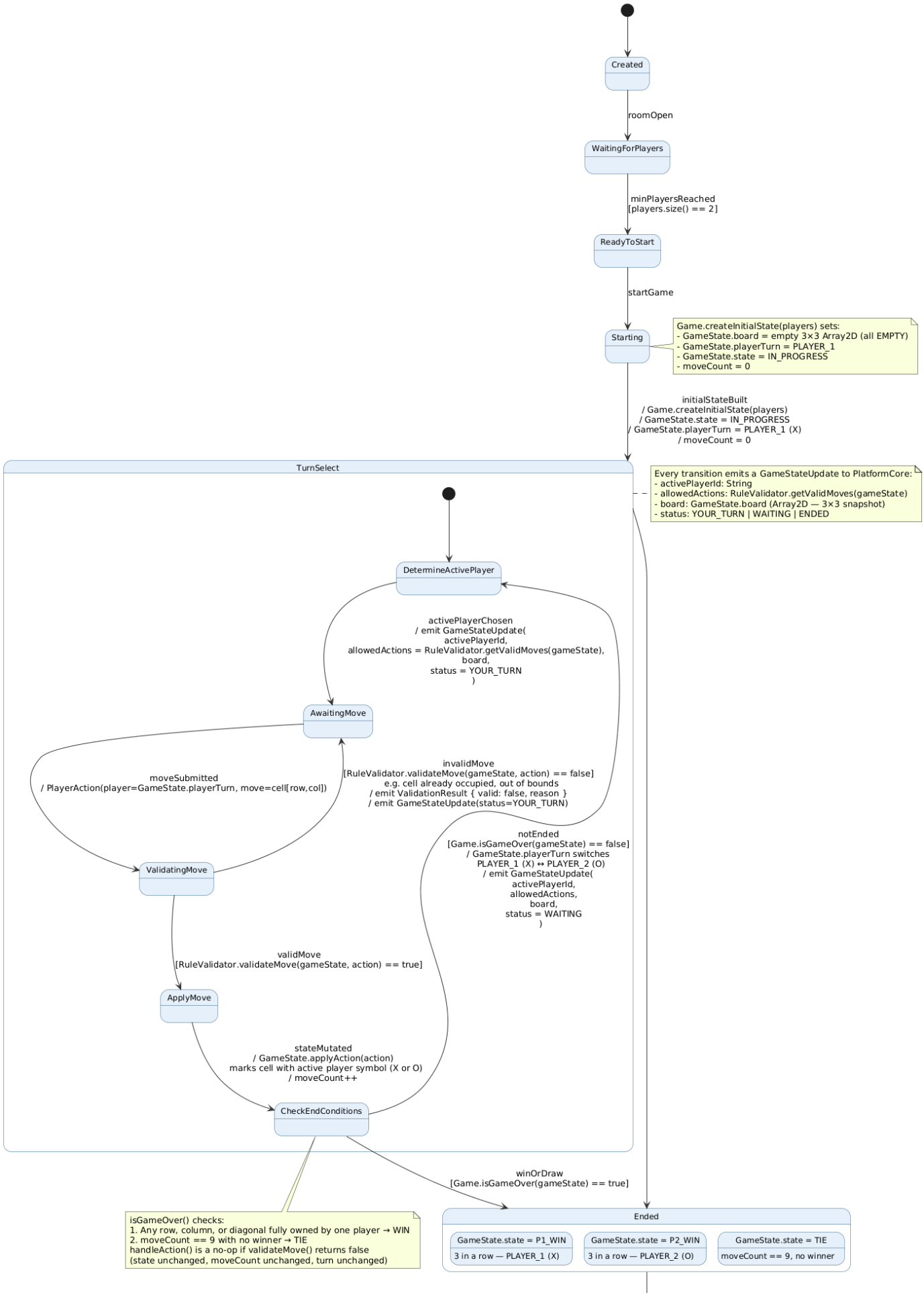
### Architectural Note

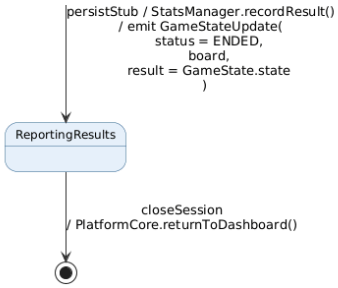
> GameSession emits `GameStateUpdate` events. PlatformCore routes notifications to Lobby/Home/Dashboard or the in-game view, even if the user is not on the game screen.

This is the key platform-first principle: notifications are platform-owned, not game-owned.

## Diagram 03 — Abstract Game State Machine

**File:** Game SM.png


Tic-Tac-Toe — State Machine
Created
roomOpen
WaitingForPlayers
minPlayersReached
[players.size() == 2]
ReadyToStart
startGame
Starting
Game.createInitialState(players) sets:
- GameState.board = empty 3×3 Array2D (all EMPTY)
- GameState.playerTurn = PLAYER_1
- GameState.state = IN_PROGRESS
- moveCount = 0
initialStateBuilt
/ Game.createInitialState(players)
/ GameState.state = IN_PROGRESS
/ GameState.playerTurn = PLAYER_1 (X)
/ moveCount = 0
TurnSelect
Every transition emits a GameStateUpdate to PlatformCore:
- activePlayerId: String
- allowedActions: RuleValidator.getValidMoves(gameState)
- board: GameState.board (Array2D — 3×3 snapshot)
- status: YOUR_TURN | WAITING | ENDED
DetermineActivePlayer
activePlayerChosen
/ emit GameStateUpdate(
activePlayerId,
allowedActions = RuleValidator.getValidMoves(gameState),
board,
status = YOUR_TURN
)
AwaitingMove
moveSubmitted
/ PlayerAction(player=GameState.playerTurn, move=cell[row,col])
invalidMove
[RuleValidator.validateMove(gameState, action) == false]
e.g. cell already occupied, out of bounds
/ emit ValidationResult { valid: false, reason }
/ emit GameStateUpdate(status=YOUR_TURN)
ValidatingMove
notEnded
[Game.isGameOver(gameState) == false]
/ GameState.playerTurn switches
PLAYER_1 (X) ↔ PLAYER_2 (O)
/ emit GameStateUpdate(
activePlayerId,
allowedActions,
board,
status = WAITING
)
validMove
[RuleValidator.validateMove(gameState, action) == true]
ApplyMove
stateMutated
/ GameState.applyAction(action)
marks cell with active player symbol (X or O)
/ moveCount++
CheckEndConditions
winOrDraw
[Game.isGameOver(gameState) == true]
isGameOver() checks:
1. Any row, column, or diagonal fully owned by one player → WIN
2. moveCount == 9 with no winner → TIE
handleAction() is a no-op if validateMove() returns false
(state unchanged, moveCount unchanged, turn unchanged)
Ended
GameState.state = P1_WIN
3 in a row — PLAYER_1 (X)
GameState.state = P2_WIN
3 in a row — PLAYER_2 (O)
GameState.state = TIE
moveCount == 9, no winner




## Purpose

Defines the complete state model shared by all concrete game implementations. Updated from the original simplified diagram to follow the full session lifecycle structure of the Game Session SM (Diagram 03) and to reference exact method and field names from the `Game`, `RuleValidator`, `GameState`, `PlayerAction`, `Player`, and `State` class diagrams.

## Session Lifecycle States

| State | Description |
|---|---|
| `Created` | Room instantiated by `GameSessionManager`. |
| `WaitingForPlayers` | Room open; waiting for minimum required players to join. |
| `ReadyToStart` | `players.size() >= minRequired`; game can be started. |
| `Starting` | `startGame` triggered; `Game.createInitialState(players)` called by subclass. Sets `GameState.state = IN_PROGRESS`, `GameState.playerTurn = PLAYER_1`. |
| `ReportingResults` | `StatsManager.recordResult()` called; `GameStateUpdate(status=ENDED)` emitted; session closes. |

## TurnSelect Composite — Internal States

| State | Description |
|---|---|
| `DetermineActivePlayer` | Selects which player acts next; emits `GameStateUpdate(status=YOUR_TURN)` with `allowedActions = RuleValidator.getValidMoves(gameState)`. |
| `AwaitingMove` | Waiting for `PlayerAction(player=GameState.playerTurn, move)` to arrive from the active player. |
| `ValidatingMove` | `RuleValidator.validateMove(gameState, action)` called. If false: `handleAction()` is a no-op — state, turn, and board are all unchanged; error emitted to active player. |
| `ApplyMove` | `GameState.applyAction(action)` called by subclass implementation. Board mutated. |
| `CheckEndConditions` | `Game.isGameOver(gameState)` evaluated. If false: `GameState.playerTurn` switches `PLAYER_1 ↔ PLAYER_2`, loop continues. If true: `GameState.state` |

| State | Description |
| --- | --- |
| | set to `P1_WIN`, `P2_WIN`, or `TIE`. |
| `Ended` | Terminal game condition reached. `GeneralStats.win() / lose() / tie()`, `updateMMR(win)`, `updateRank()` called for all players. |

## Key Transitions

| From | Event / Guard / Action | To |
| --- | --- | --- |
| *(initial)* | `roomOpen` | `Created` |
| `Created` | `roomOpen` | `WaitingForPlayers` |
| `WaitingForPlayers` | `minPlayersReached [players.size() >= minRequired]` | `ReadyToStart` |
| `ReadyToStart` | `startGame` | `Starting` |
| `Starting` | `initialStateBuilt / Game.createInitialState(players)` | `TurnSelect → DetermineActivePlayer` |
| `DetermineActivePlayer` | `activePlayerChosen` / emit `GameStateUpdate(YOUR_TURN)` | `AwaitingMove` |
| `AwaitingMove` | `moveSubmitted / PlayerAction(player, move)` | `ValidatingMove` |
| `ValidatingMove` | `[RuleValidator.validateMove() == false]` / no-op, emit error | `AwaitingMove` (self-loop) |
| `ValidatingMove` | `[RuleValidator.validateMove() == true]` | `ApplyMove` |
| `ApplyMove` | `stateMutated / GameState.applyAction(action)` | `CheckEndConditions` |
| `CheckEndConditions` | `[Game.isGameOver() == false]` / playerTurn switches, emit `GameStateUpdate(WAITING)` | `DetermineActivePlayer` |
| `CheckEndConditions` | `[Game.isGameOver() == true]` / set `GameState.state` | `Ended` |
| `Ended` | `persistStub / StatsManager.recordResult()` / emit `GameStateUpdate(ENDED)` | `ReportingResults` |
| `ReportingResults` | `closeSession / PlatformCore.returnToDashboard()` | *(final state)* |

### GameStateUpdate Contract

Every transition emits a `GameStateUpdate` to `PlatformCore` containing:

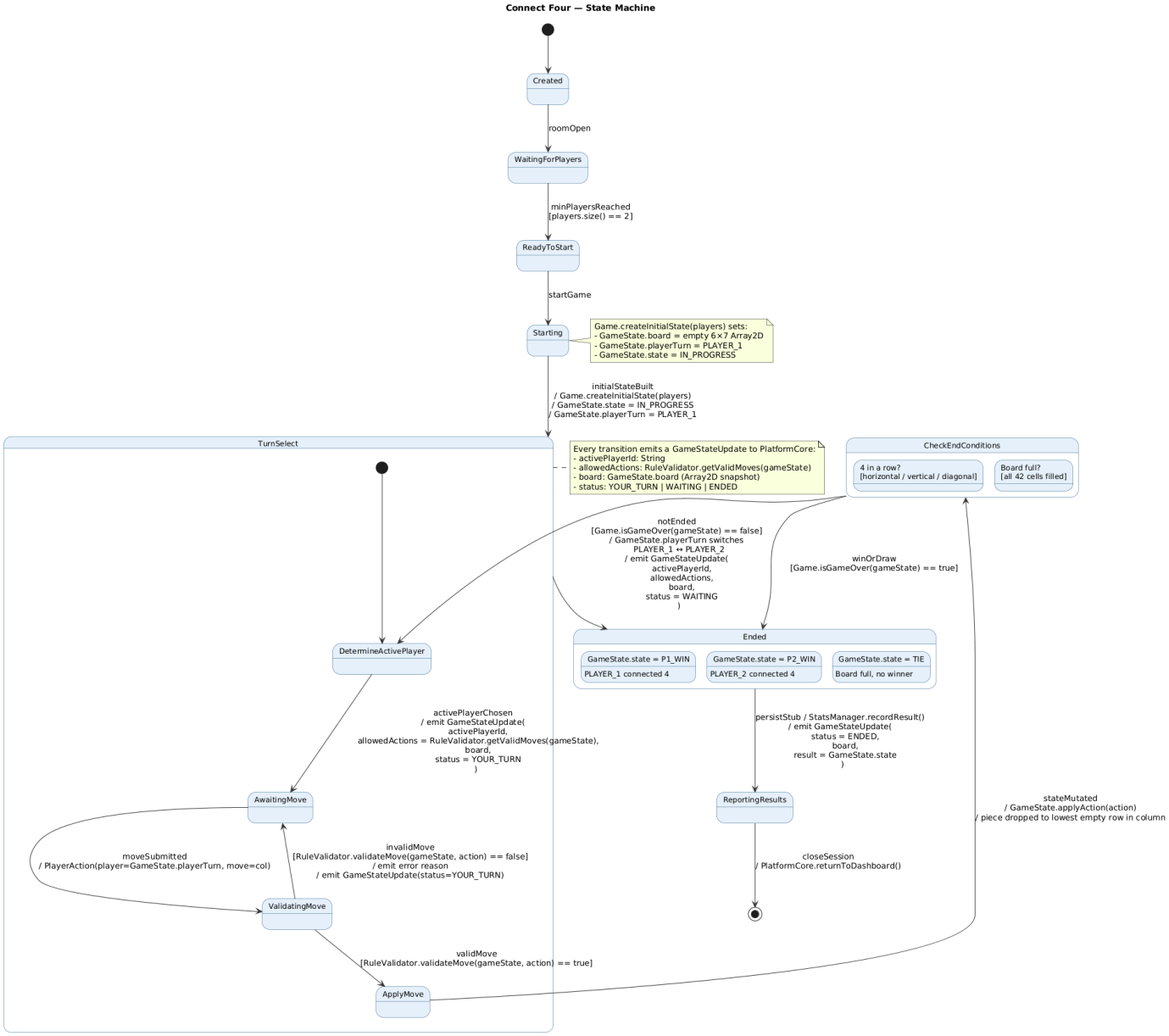
| Field | Value |
|---|---|
| `activePlayerId` | `GameState.playerTurn` |
| `allowedActions` | `RuleValidator.getValidMoves(gameState)` |
| `board` | `GameState.board` (Array2D snapshot or delta) |
| `status` | `YOUR_TURN` / `WAITING` / `ENDED` |

### Class Diagram Traceability

| SM Element | Class Diagram Source |
|---|---|
| `Game.createInitialState(players)` | `Game` abstract class |
| `Game.handleAction(action)` | `Game` abstract class |
| `Game.isGameOver(state)` | `Game` abstract class |
| `RuleValidator.validateMove(state, move)` | `RuleValidator` abstract class |
| `RuleValidator.getValidMoves(state)` | `RuleValidator` abstract class |
| `GameState.applyAction(action)` | `GameState` abstract class |
| `GameState.board` , `playerTurn` , `state` | `GameState` fields |
| `PlayerAction(player, move)` | `PlayerAction` class |
| `PLAYER_1` , `PLAYER_2` | `Player` enum |
| `IN_PROGRESS` , `P1_WIN` , `P2_WIN` , `TIE` | `State` enum |
| `GeneralStats.win()` , `lose()` , `tie()` , `updateMMR()` , `updateRank()` | `GeneralStats` class (Diagram 02) |

# Diagram 04 — Connect Four State Machine

**File:** `Connect 4 SM.png`




## Purpose

Concrete state machine for the Connect Four game implementation. Updated to include the full session lifecycle (matching Diagrams 03 and 04) and to reference exact class diagram method names. Extends the abstract structure with Connect Four-specific terminal conditions and board initialization.

## Session Lifecycle States

| State | Description |
|---|---|
| `Created` | Room instantiated by `GameSessionManager`. |
| `WaitingForPlayers` | Waiting for exactly 2 players ( `players.size() == 2` ). |
| `ReadyToStart` | Both players present; game can start. |

| State | Description |
| --- | --- |
| `Starting` | `Game.createInitialState(players)` initializes a 6×7 `Array2D` board, sets `GameState.playerTurn = PLAYER_1` , `GameState.state = IN_PROGRESS` . |
| `ReportingResults` | `StatsManager.recordResult()` and `GeneralStats` updates called; `GameStateUpdate(ENDED)` emitted; session closes. |

## TurnSelect Composite — Internal States

| State | Description |
| --- | --- |
| `DetermineActivePlayer` | Determines whose turn it is; emits `GameStateUpdate(YOUR_TURN)` with `allowedActions = RuleValidator.getValidMoves(gameState)` (non-full columns). |
| `AwaitingMove` | Waiting for `PlayerAction(player=GameState.playerTurn, move=col)` — column selection. |
| `ValidatingMove` | `RuleValidator.validateMove(gameState, action)` checks column is in bounds and not full. If false: no-op, error emitted, player retries. |
| `ApplyMove` | `GameState.applyAction(action)` drops piece to the lowest empty row in the selected column. |
| `CheckEndConditions` | `Game.isGameOver(gameState)` checks for 4-in-a-row (horizontal, vertical, diagonal) or full board. `playerTurn` alternates `PLAYER_1 ↔ PLAYER_2` on continuation. |
| `Ended` | Terminal reached. `GameState.state` is `P1_WIN` , `P2_WIN` , or `TIE` . Stats updated via `GeneralStats` . |

## Key Transitions

| From | Event / Guard / Action | To |
| --- | --- | --- |
| *(initial)* | `roomOpen` | `Created` |
| `Created` | `roomOpen` | `WaitingForPlayers` |
| `WaitingForPlayers` | `minPlayersReached [players.size() == 2]` | `ReadyToStart` |
| `ReadyToStart` | `startGame` | `Starting` |
| `Starting` | `initialStateBuilt` / board = empty 6×7 Array2D, `playerTurn = PLAYER_1` | `DetermineActivePlayer` |
| `DetermineActivePlayer` | `activePlayerChosen` / emit `GameStateUpdate(YOUR_TURN,` | `AwaitingMove` |

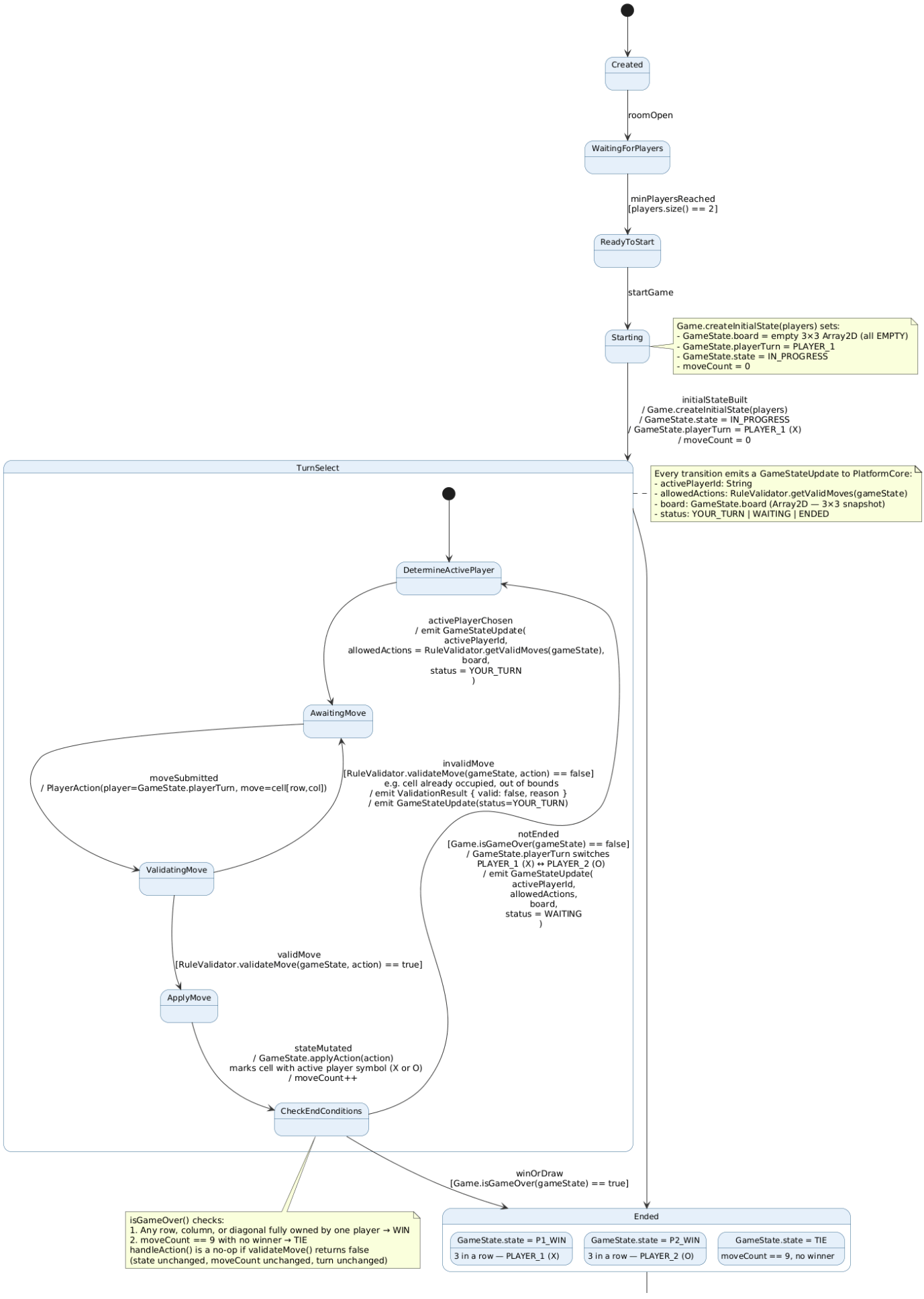
| From | Event / Guard / Action | To |
|---|---|---|
| | allowedActions=non-full cols) | |
| AwaitingMove | moveSubmitted / PlayerAction(player, move=col) | ValidatingMove |
| ValidatingMove | [validateMove() == false] (col full or out of bounds) / no-op, emit error | AwaitingMove |
| ValidatingMove | [validateMove() == true] | ApplyMove |
| ApplyMove | stateMutated / GameState.applyAction(action) — piece placed at lowest empty row | CheckEndConditions |
| CheckEndConditions | [isGameOver() == false, valid move, board not full] / playerTurn switches | DetermineActivePlayer |
| CheckEndConditions | [isGameOver() == true, PLAYER_1 gets 4 in a row] / GameState.state = P1_WIN | Ended |
| CheckEndConditions | [isGameOver() == true, PLAYER_2 gets 4 in a row] / GameState.state = P2_WIN | Ended |
| CheckEndConditions | [isGameOver() == true, board full, no winner] / GameState.state = TIE | Ended |
| Ended | persistStub / StatsManager.recordResult() | ReportingResults |
| ReportingResults | closeSession / PlatformCore.returnToDashboard() | *(final state)* |

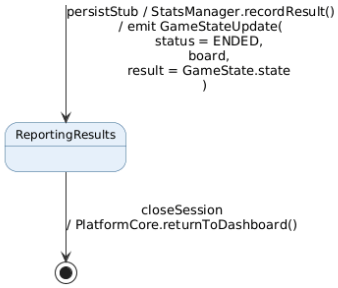
### Connect Four–Specific Behavior

- `PlayerAction.move` is a column index (0–6); `applyAction()` resolves the row by gravity (lowest empty cell).
- `RuleValidator.getValidMoves(gameState)` returns only non-full columns as allowed actions.
- `isGameOver()` checks all four directions (horizontal, vertical, diagonal both ways) for a sequence of 4 owned by one player, then falls back to checking whether all 42 cells are filled for the TIE condition.
- Turn alternation is a side-effect of `CheckEndConditions → DetermineActivePlayer`, not an internal `applyAction()` concern.

## Diagram 05 — Tic-Tac-Toe State Machine

**File:** 02.03 – Tic Tac Toe SM Diagram.png


Tic-Tac-Toe — State Machine
Created
roomOpen
WaitingForPlayers
minPlayersReached
[players.size() == 2]
ReadyToStart
startGame
Starting
Game.createInitialState(players) sets:
- GameState.board = empty 3×3 Array2D (all EMPTY)
- GameState.playerTurn = PLAYER_1
- GameState.state = IN_PROGRESS
- moveCount = 0
initialStateBuilt
/ Game.createInitialState(players)
/ GameState.state = IN_PROGRESS
/ GameState.playerTurn = PLAYER_1 (X)
/ moveCount = 0
TurnSelect
Every transition emits a GameStateUpdate to PlatformCore:
- activePlayerId: String
- allowedActions: RuleValidator.getValidMoves(gameState)
- board: GameState.board (Array2D — 3×3 snapshot)
- status: YOUR_TURN | WAITING | ENDED
DetermineActivePlayer
activePlayerChosen
/ emit GameStateUpdate(
activePlayerId,
allowedActions = RuleValidator.getValidMoves(gameState),
board,
status = YOUR_TURN
)
AwaitingMove
moveSubmitted
/ PlayerAction(player=GameState.playerTurn, move=cell[row,col])
invalidMove
[RuleValidator.validateMove(gameState, action) == false]
e.g. cell already occupied, out of bounds
/ emit ValidationResult { valid: false, reason }
/ emit GameStateUpdate(status=YOUR_TURN)
ValidatingMove
notEnded
[Game.isGameOver(gameState) == false]
/ GameState.playerTurn switches
PLAYER_1 (X) ↔ PLAYER_2 (O)
/ emit GameStateUpdate(
activePlayerId,
allowedActions,
board,
status = WAITING
)
validMove
[RuleValidator.validateMove(gameState, action) == true]
ApplyMove
stateMutated
/ GameState.applyAction(action)
marks cell with active player symbol (X or O)
/ moveCount++
CheckEndConditions
winOrDraw
[Game.isGameOver(gameState) == true]
isGameOver() checks:
1. Any row, column, or diagonal fully owned by one player → WIN
2. moveCount == 9 with no winner → TIE
handleAction() is a no-op if validateMove() returns false
(state unchanged, moveCount unchanged, turn unchanged)
Ended
GameState.state = P1_WIN
3 in a row — PLAYER_1 (X)
GameState.state = P2_WIN
3 in a row — PLAYER_2 (O)
GameState.state = TIE
moveCount == 9, no winner




## Purpose

Concrete state machine for the Tic-Tac-Toe game implementation. Updated to include the full session lifecycle, use consistent `State` enum naming ( `P1_WIN` / `P2_WIN` / `TIE` instead of the original `WIN` / `DRAW` ), and reference exact class diagram method names. Retains the `moveCount` tracking from the original as a game-specific initialization detail.

## Session Lifecycle States

| State | Description |
|---|---|
| `Created` | Room instantiated by `GameSessionManager` . |
| `WaitingForPlayers` | Waiting for exactly 2 players ( `players.size() == 2` ). |
| `ReadyToStart` | Both players present; game can start. |
| `Starting` | `Game.createInitialState(players)` initializes a 3×3 `Array2D` board (all `EMPTY` ), sets `GameState.playerTurn = PLAYER_1` (X), `GameState.state = IN_PROGRESS` , `moveCount = 0` . |
| `ReportingResults` | `StatsManager.recordResult()` and `GeneralStats` updates called; `GameStateUpdate(ENDED)` emitted with `GameState.state` ; session closes. |

## TurnSelect Composite — Internal States

| State | Description |
|---|---|
| `DetermineActivePlayer` | Determines whose turn it is (X or O); emits `GameStateUpdate(YOUR_TURN)` with `allowedActions = RuleValidator.getValidMoves(gameState)` (all `EMPTY` cells). |
| `AwaitingMove` | Waiting for `PlayerAction(player=GameState.playerTurn, move=cell[row,col])` . |
| `ValidatingMove` | `RuleValidator.validateMove(gameState, action)` checks cell is `EMPTY` and within 3×3 bounds. If false: `handleAction()` is a no-op — `moveCount` unchanged, turn unchanged; `ValidationResult { valid: false }` emitted. |
| `ApplyMove` | `GameState.applyAction(action)` marks cell with active player symbol (X or O). `moveCount++` . |

| State | Description |
| --- | --- |
| `CheckEndConditions` | `Game.isGameOver(gameState)` checks for 3-in-a-row in any row, column, or diagonal, then checks `moveCount == 9` for draw. `playerTurn` switches `PLAYER_1 (X) ↔ PLAYER_2 (O)` on continuation. |
| `Ended` | Terminal reached. `GameState.state` is `P1_WIN`, `P2_WIN`, or `TIE`. Stats updated via `GeneralStats`. |

## Key Transitions

| From | Event / Guard / Action | To |
| --- | --- | --- |
| *(initial)* | `roomOpen` | `Created` |
| `Created` | `roomOpen` | `WaitingForPlayers` |
| `WaitingForPlayers` | `minPlayersReached [players.size() == 2]` | `ReadyToStart` |
| `ReadyToStart` | `startGame` | `Starting` |
| `Starting` | `initialStateBuilt` / board = empty 3×3 Array2D, `playerTurn = PLAYER_1`, `moveCount = 0` | `DetermineActivePlayer` |
| `DetermineActivePlayer` | `activePlayerChosen` / emit `GameStateUpdate(YOUR_TURN, allowedActions=EMPTY cells)` | `AwaitingMove` |
| `AwaitingMove` | `moveSubmitted / PlayerAction(player, move=cell[row,col])` | `ValidatingMove` |
| `ValidatingMove` | `[validateMove() == false]` (cell occupied or out of bounds) / no-op, `moveCount` unchanged, emit `ValidationResult { valid: false }` | `AwaitingMove` |
| `ValidatingMove` | `[validateMove() == true]` | `ApplyMove` |
| `ApplyMove` | `stateMutated / GameState.applyAction(action)` — cell marked X or O / `moveCount++` | `CheckEndConditions` |
| `CheckEndConditions` | `[isGameOver() == false] / playerTurn` switches `PLAYER_1 ↔ PLAYER_2` | `DetermineActivePlayer` |
| `CheckEndConditions` | `[isGameOver() == true, 3 in a row, PLAYER_1] / GameState.state = P1_WIN` | `Ended` |

| From | Event / Guard / Action | To |
| --- | --- | --- |
| `CheckEndConditions` | `[isGameOver() == true, 3 in a row, PLAYER_2] / GameState.state = P2_WIN` | `Ended` |
| `CheckEndConditions` | `[isGameOver() == true, moveCount == 9, no winner] / GameState.state = TIE` | `Ended` |
| `Ended` | `persistStub / StatsManager.recordResult()` | `ReportingResults` |
| `ReportingResults` | `closeSession / PlatformCore.returnToDashboard()` | *(final state)* |

## Tic-Tac-Toe–Specific Behavior

- `PlayerAction.move` is a cell reference `[row, col]` on the 3×3 board; `applyAction()` places the active player's symbol at that cell.
- `RuleValidator.getValidMoves(gameState)` returns all cells where `GameState.board[row][col] == EMPTY`.
- `moveCount` is initialized to 0 in `Starting` and incremented inside `ApplyMove`. The `TIE` guard `moveCount == 9` is evaluated inside `isGameOver()` after confirming no 3-in-a-row exists.
- `handleAction()` is fully rejected (no-op) when `validateMove()` returns false — `moveCount`, `playerTurn`, and board state are all unchanged.
- Naming aligned to `State` enum: `P1_WIN` / `P2_WIN` / `TIE` replace the original diagram's `WIN` / `DRAW` labels for consistency with the abstract SM and class diagram.

| Event | Emitted by | Consumed by |
| --- | --- | --- |
| `GameStateUpdate(opponentTurn / yourTurn / gameOver)` | Game Session SM (Diagram 03) | Player Session SM (Diagrams 01 & 02) |
| `submitMove` | Player Session SM (via UI) | Game Session SM → `ValidatingMove` |
| `PlayerAction(player, move)` | Game Session SM's `AwaitingMove` state | `RuleValidator.validateMove()` / `GameState.applyAction()` |
| `RuleValidator.validateMove(gameState, action)` | `ValidatingMove` state | Returns boolean to `CheckEndConditions` guard |
| `GameState.applyAction(action)` | `ApplyMove` state | Mutates `GameState.board`, sets `GameState.state` |
| `Game.isGameOver(gameState)` | `CheckEndConditions` state | Returns boolean; drives terminal vs. continuation branch |
| `GameState.state` ( `P1_WIN` / `P2_WIN` / `TIE` ) | Concrete Game SM ( `applyAction` ) | Game Session SM → `Ended` |

| Event | Emitted by | Consumed by |
| --- | --- | --- |
| `StatsManager.recordResult()` | Game Session SM's `Ended` state | `GeneralStats.win()` / `lose()` / `tie()` / `updateMMR()` / `updateRank()` |
| `PlatformCore.returnToDashboard()` | `ReportingResults` state | Player Session SM → `Browsing` |